

Algorithme de Boyer-Moore-Horspool

Trouver les occurrences d'une chaîne de caractères (motif ou mot) dans un texte est un problème qui se rencontre fréquemment dans les éditeurs de texte. Les algorithmes de recherche de chaînes de caractères sont également utilisés pour trouver une séquence ADN, par exemple, ou pour rechercher des pages web correspondantes à une recherche Internet.

Exemple : on souhaite rechercher le mot `exo` dans le texte : « Bonjour Je fais l'exo de NSI »

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot	e	x	o																									

1. Programmation naïve

a) Principe

On compare le B et le e : il n'y a pas de correspondance, alors on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot	e	x	o																									

On compare le o et le e : il n'y a pas de correspondance, alors on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot		e	x	o																								

On compare le n et le e : il n'y a pas de correspondance, alors on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot			e	x	o																							

.....
On fait de même jusqu'au premier e

On compare le caractère « espace » et le x : il n'y a pas de correspondance, on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot										e	x	o																

On compare le caractère « espace » et le e : il n'y a pas de correspondance, on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot											e	x	o															

On compare le f et le e : il n'y a pas de correspondance, alors on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot												e	x	o														

.....
On fait de même jusqu'au prochain e.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																			e	x	o							

On compare le x et le x, comme il y a correspondance, on compare le o et le o.

Il y a trois correspondances correspondant à la taille du mot donc nous avons trouvé une occurrence.

.....
On se décale d'un caractère jusqu'au prochain e.

On compare le caractère « espace » et le x : il n'y a pas de correspondance, on décale d'un caractère

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																								e	x	o		

On compare le caractère « espace » et le e : il n'y a pas de correspondance, on décale d'un caractère

i	0	1	2	3	4		5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o		u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																										e	x	o	

On compare le N et le e : il n'y a pas de correspondance, on arrête la recherche et on renvoie 1 occurrence.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																										e	x	o

Algorithme de Boyer-Moore-Horspool

b) Algorithme

L'algorithme de cette méthode naïve est le suivant :

Algorithme 1 : recherche_naive(motif, texte)

```
1:  $n \leftarrow$  longueur du texte
2:  $m \leftarrow$  longueur du motif
3:  $\text{compteur\_occurrences} \leftarrow 0$ 
4: Pour  $s$  de 0 à  $n - m$  faire
5:   Si  $\text{motif}[0 \dots m] == \text{texte}[s \dots s + m]$  alors
6:      $\text{compteur\_occurrences} \leftarrow \text{compteur\_occurrences} + 1$ 
7:   Fin Si
8: Fin Pour
9: Renvoyer  $\text{compteur\_occurrences}$ 
```

c) Codage

Ouvrir le fichier TP-Boyer-Moore.py et écrire le script de la fonction recherche_naive : cette fonction prend pour paramètres motif et texte, deux chaînes de caractères, et renvoie le nombre d'occurrences de motif dans texte.

Une assertion vérifiera que le texte est de longueur supérieure ou égale à celle du motif.

On pourra aussi proposer un affichage du décalage de chaque occurrence trouvée.

Tester avec le texte : « Bonjour, je fais l'exo de NSI » et le motif « exo » (donnés par l'utilisateur grâce à deux appels input).

```
*** Console de processus distant Réinitialisée ***
L'occurrence 1 à été trouvée au 19 ième caractère
En tout, il y a : 1 occurrence(s)
```

2. Algorithme de Boyer Moore Horspool

L'algorithme de Boyer-Moore-Horspool ou Horspool est un algorithme de recherche de sous-chaîne publié en 1980 par Nigel Horspool, un professeur à l'université de Victoria au Canada.

Il consiste en une simplification de l'algorithme de Boyer-Moore.

a) Principe

L'algorithme de Boyer-Moore effectue la vérification, c'est-à-dire qu'il tente d'établir la correspondance entre le motif et le texte à une certaine position, à l'envers.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot	e	x	o																									

On va comparer le o et le n.

Comme le n n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot				e	x	o																						

On va comparer le o et le u.

Comme le u n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot							e	x	o																			

On va comparer le o et le J.

Comme le J n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot										e	x	o																

On va comparer le o et le f.

Comme le f n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot													e	x	o													

On va comparer le o et le s.

Comme le s n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères)

Algorithme de Boyer-Moore-Horspool

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																e	x	o										

On va comparer le o et le '.

Comme le ' n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																			e	x	o							

On va comparer le o et le o.

Comme il y a correspondance, on va comparer le x et le x et, comme il y a encore correspondance, on va comparer le e et le e (là on a 3 correspondances qui correspondent à la taille du motif) on va donc ajouter 1 au compteur d'occurrences puis faire un saut de trois caractères.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	E	x	o		d	e		N	S	I
mot																					e	x	o					

On va comparer le o et le e.

Ici e est dans le motif donc on ne peut pas décaler de 3 caractères mais seulement faire correspondre les deux e (longueur motif - indice du e dans le motif + 1 : $3-0-1=2$) soit un décalage de 2 caractères.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Texte	B	o	n	j	o	u	r		J	e		f	a	i	s		l	'	e	x	o		d	e		N	S	I
mot																							e	x	o			

On va comparer le o et le N. Comme le N n'est pas dans le motif, on peut faire un saut de la taille du motif (ici 3 caractères) mais $25+3>27$ donc on peut arrêter de chercher, il ne peut plus y avoir d'occurrences.

On se rend compte qu'avec cette méthode, on fait des sauts plus grands qu'avec la méthode naïve. Ce principe de fonctionnement explique pourquoi plus le motif est long, et plus l'algorithme est efficace pour le trouver.

b) Cas particulier lorsque certaines cases coïncident

i	0	1	2	3	4	5	6	7	8	9	10	11
Texte	C	T	G	T	G	A	G	C	A	T	G	A
mot	C	T	G	C	G	A						

Des cas plus complexes peuvent se produire lorsque certaines lettres coïncident, car le saut doit être calculé par rapport à l'indice de la lettre du motif.

Par exemple, si les deux lettres G et A coïncident, on teste celle en 4e position (indice 3) qui ne coïncide pas puisqu'on lit un T au lieu d'un C.

Le T est dans le motif, on va effectuer un saut de 2, résultat obtenu par la succession des deux opérations :

- longueur motif - indice du T dans le motif - 1 : $6 - 1 - 1 = 4$
- puis, on enlève 2 car on a deux caractères qui coïncident : $4 - 2 = 2$.

i	0	1	2	3	4	5	6	7	8	9	10	11
Texte	C	T	G	T	G	A	G	C	A	T	G	A
mot			C	T	G	C	G	A				

On va superposer les deux premiers C :

- longueur motif - indice du C dans le motif - 1 : $6 - 3 - 1 = 2$

Donc, on fait un décalage de 2 caractères, ce qui est possible puisque $7 + 2 = 9 \leq 11$.

i	0	1	2	3	4	5	6	7	8	9	10	11
Texte	C	T	G	T	G	A	G	C	A	T	G	A
mot				C	T	G	C	G	A			

On va superposer les deux premiers T :

- longueur motif - indice du T dans le motif - 1 : $6 - 1 - 1 = 4$.

Donc, on doit faire un décalage de 4 caractères mais cela est impossible puisque $9 + 4 = 13 > 11$.

Il n'y a donc pas d'occurrence dans cet exemple.

Pour trouver ce saut pour chaque lettre, on remarque qu'il suffit de regarder l'indice de cette lettre dans le motif (lettres numérotées de 0 à $\text{len}(\text{motif})-1$) et de calculer $\text{len}(\text{motif})-\text{imotif}-1$. Si une lettre apparaît plusieurs fois, on prendra la dernière valeur lors du parcours du motif (elle correspondra à la plus petite).

Ceci correspondra à la **table des sauts** qu'il faudra faire en prétraitement.

Algorithme de Boyer-Moore-Horspool

c) Codage de la table des sauts

On va construire ce que l'on nomme une **table des sauts**, pour chaque caractère du motif. Cette table traduit en fait l'écart minimal entre une lettre du motif et la fin du motif. La dernière lettre du mot est traitée à part, elle renvoie un écart maximal si elle n'est pas présente ailleurs dans le mot.

Exemples

Motif	CTGCGA			
Lettres	A et les autres lettres	G	C	T
Distance à la fin du motif	6	1	2	4

Motif	ACTGCGA				
Lettres	Les autres lettres	G	C	T	A
Distance à la fin du motif	7	1	2	4	6

Compléter le fichier `TP-Boyer-Moore.py` en écrivant le script de la fonction `table_sauts` qui prend comme paramètre une chaîne de caractère `motif`. L'algorithme en pseudo-code est donné ci-dessous :

Affecter à la variable `table` un dictionnaire vide

Affecter à la variable `i` la valeur 0

Pour chaque lettre du motif sans le dernier caractère :

Affecter à la valeur de la clé lettre du dictionnaire la taille du motif – `i` – 1

Incrémenter `i` de 1

Renvoyer le dictionnaire `table`.

Tester avec les motifs `exo`, `CTGCGA` et `ACTGCGA`.

```
Table de sauts pour le motif exo: {'x': 1, 'e': 2}
Table de sauts pour le motif CTGCGA: {'T': 4, 'C': 2, 'G': 1}
Table de sauts pour le motif ACTGCGA: {'T': 4, 'A': 6, 'C': 2, 'G': 1}
```

d) Codage de la fonction Boyer-Moore-Horspool

Ouvrir le fichier `Fonction_boyer_moore_horspool-et-tests.py`: copier-coller le script de la fonction dans votre fichier `TP-Boyer-Moore.py` et compléter tous les commentaires après les symboles `#`.

Tester avec le texte : « Bonjour, je fais l'exo de NSI » et le motif « `exo` » (donnés par l'utilisateur grâce à deux appels `input`).

```
*** Console de processus distant Réinitialisée ***
[19]
...
```

3. Efficacité de l'algorithme de Boyer Moore Horspool

a) Calcul de comparaisons effectuées par la méthode naïve

Modifier la fonction `recherche_naive` afin d'afficher le nombre de comparaisons lors de l'appel de celle-ci. Tester avec le texte : « Bonjour, je fais l'exo de NSI » et le motif « `exo` » (donnés par l'utilisateur grâce à deux appels `input`).

b) Calcul de comparaisons effectuées par la méthode Boyer Moore Horspool

Modifier la fonction `boyer_moore_horspool` afin d'afficher le nombre de comparaisons lors de l'appel de celle-ci. Tester avec le texte : « Bonjour, je fais l'exo de NSI » et le motif « `exo` » (donnés par l'utilisateur grâce à deux appels `input`).

```
*** Console de processus distant Réinitialisée ***
L'occurrence 1 à été trouvée au 19 ième caractère
Avec la méthode naïve, on a fait : 27 comparaison(s)
Avec la méthode Boyer Moore Horspool, on a fait : 13 comparaison(s)
```

Algorithme de Boyer-Moore-Horspool

c) Comparaison des deux algorithmes avec un texte plus long

On va comparer les deux algorithmes sur un texte plus long en faisant varier le motif.

Pour cela, on va utiliser le fichier texte `texte_comparaison.txt` : il s'agit du texte intégral du roman Le Rouge et Le Noir de Stendhal.

Pour lire le fichier et le stocker dans la variable `texte`, écrire les deux instructions suivantes :

```
fichier_texte = open("Texte_comparaison.txt", "r", encoding="UTF-8")
texte = fichier_texte.read()
```

Tester les motifs suivants (on affichera le nombre d'occurrences et de comparaison pour chaque méthode) :

- e
- lorsque,
- extraordinaire
- alors, je
- Cet homme de tant d'esprit avait l'air inquiet

Conclure sur l'efficacité de l'algorithme suivant la taille du motif.

4. Remarques finales

Complexité

La complexité d'un algorithme de recherche textuelle se mesure essentiellement par le nombre d'opérations de comparaison de caractères qu'il effectue. L'analyse précise de la complexité de l'algorithme que nous avons programmé est difficile, et dépasse le niveau attendu en enseignement NSI.

Simplement, signalons qu'il est considéré comme un algorithme sous-linéaire : dans des cas favorables, les décalages de la fenêtre sont de l'ordre de la taille p du motif. En fait, l'algorithme n'est même pas tenu de lire l'intégralité du texte : c'est la sous-linéarité.

Dans les cas favorables, on peut estimer un coût de l'ordre de n/p , ce qui est évidemment très intéressant.

Prétraitement

Le calcul de la **table des sauts** est effectué une seule fois pour un motif donné.

Bien sûr, un tel prétraitement a un coût, mais celui-ci ne doit être compté qu'une fois même si on effectue plusieurs recherches avec le même motif. Cette stratégie d'un prétraitement des données est un paradigme de programmation qu'on peut garder en tête pour d'autres applications.

5. Sources

- Site de Gisèle Bareux : http://gisele.bareux.free.fr/NSI1_Term.htm
- Document ressource du site Eduscol : <https://eduscol.education.fr/document/30076/download>